ÉTUDE DE CAS — STRIPE

LIVRABLE 3

Conception du schéma OLAP

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l'infrastructure de données et de calcul pour l'IA

Ce livrable présente le modèle du data warehouse de Stripe, déployé sur Amazon Redshift. Il est conçu pour l'analyse de gros volumes de données historiques : revenus, segmentation client, performance des produits, fraude et conformité. Il est alimenté toutes les heures par l'ELT décrit au livrable 5, à partir des données copiées depuis l'OLTP dans le Data Lake S3.

1. Schéma en étoile

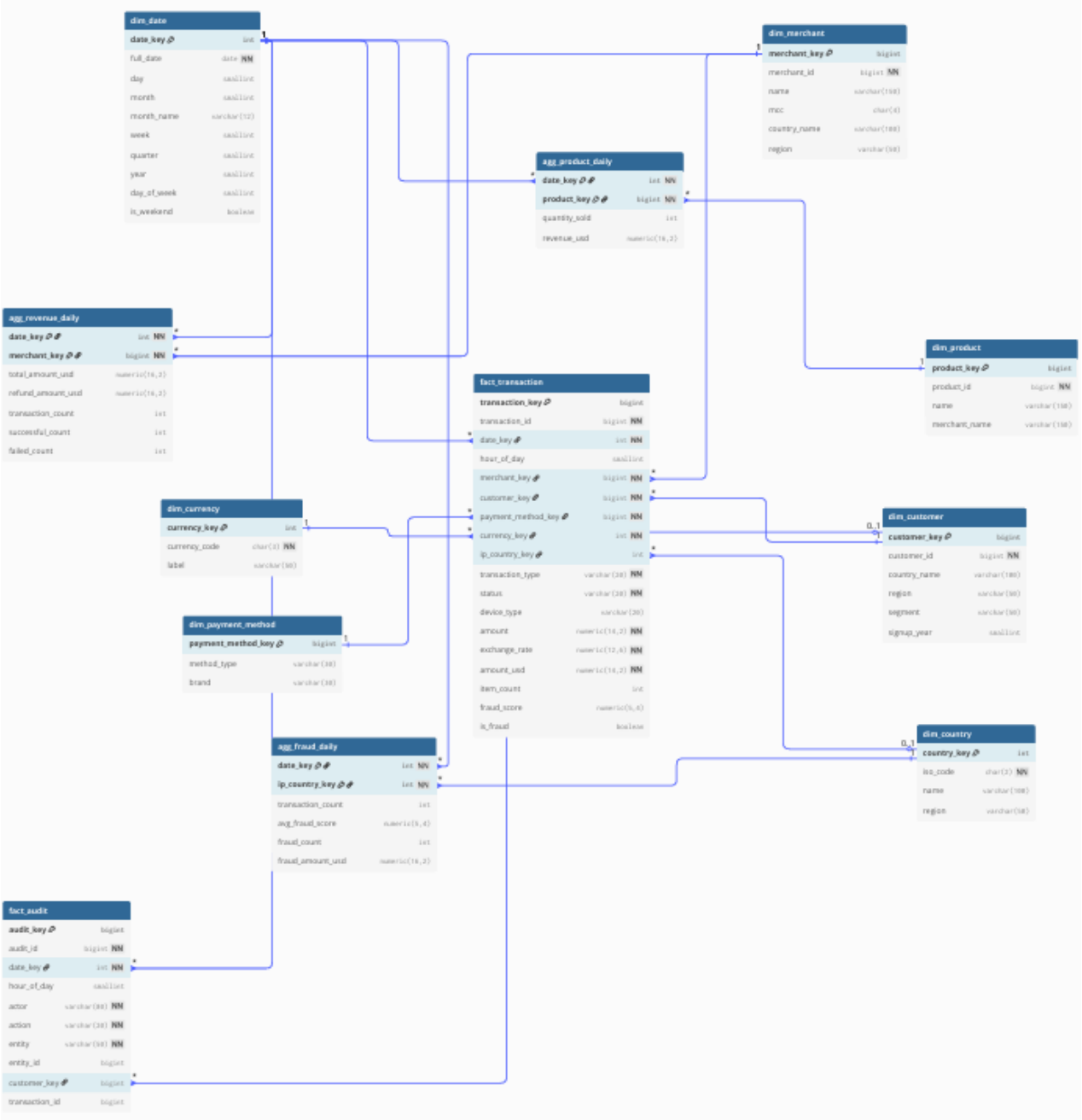


Figure 1 – Schéma en étoile du data warehouse (dbdiagram.io)

Un schéma en étoile a été choisi plutôt qu'un schéma en flocon. Dans un schéma en étoile, chaque dimension est une seule table, même si certaines informations y sont répétées (par exemple le pays et la région du marchand sont écrits directement dans dim_merchant). Les requêtes ont donc moins de jointures à faire, ce qui est plus rapide sur de gros volumes. Cette répétition n'est pas un problème ici, car le data warehouse est seulement lu par les analystes : les données ne sont modifiées que par l'ELT.

2. Tables de faits et dimensions

Table	Une ligne correspond à	Utilisation
fact_transaction	Une transaction (paiement, remboursement, rétrofacturation, abonnement)	Montants (dans la devise d'origine et en USD), nombre de produits, score de fraude, fraude confirmée (is_fraud), heure, appareil
fact_audit	Une action enregistrée dans le journal d'audit de l'OLTP	Rapports de conformité : qui a consulté ou modifié quelles données, et quand
dim_date	Un jour	Analyse par jour, semaine, mois, trimestre et année, distinction semaine / week-end
dim_merchant	Un marchand	Analyse par marchand, secteur d'activité (mcc), pays et région
dim_customer	Un client	Analyse par pays, région, segment et année d'inscription, sans donnée personnelle (ni nom, ni email, ni téléphone)
dim_payment_method	Un type de moyen de paiement	Analyse par type (carte, virement, portefeuille) et par marque
dim_currency	Une devise	Analyse par devise
dim_country	Un pays	Pays de l'adresse IP, utilisé pour l'analyse de la fraude
dim_product	Un produit	Analyse de la performance des produits

Les dimensions utilisent leur propre clé (merchant_key, customer_key…) et gardent l'identifiant de l'OLTP (merchant_id, customer_id…). Cela permet de toujours relier une ligne du data warehouse à la donnée d'origine. Le segment du client est mis à jour chaque semaine par le modèle de segmentation (livrable 7). Seul le segment actuel est conservé, car les analyses prévues n'ont pas besoin de l'historique des segments.

3. Stratégie d'agrégation

Les questions les plus fréquentes portent sur des totaux par jour, par marchand, par pays ou par produit. Plutôt que de relire à chaque fois des millions de transactions, ces totaux sont calculés à l'avance :

Objet	Contenu	Mise à jour
agg_revenue_daily (table)	Revenus, remboursements et nombre de transactions par jour et par marchand	Recalculée par l'ELT pour les jours concernés, toutes les heures
agg_fraud_daily (table)	Nombre de transactions et de fraudes, score moyen et montant fraudé par jour et par pays de l'adresse IP	Recalculée par l'ELT, toutes les heures
agg_product_daily (table)	Quantité vendue et revenus par jour et par produit, calculés à partir des lignes de transaction (transaction_item) chargées dans les tables temporaires de l'ELT	Recalculée par l'ELT, toutes les heures
mv_revenue_weekly et mv_revenue_monthly (vues matérialisées)	Revenus par semaine et par mois, par marchand, calculés à partir de agg_revenue_daily	Rafraîchies par Airflow après chaque chargement

Une vue matérialisée est une requête dont le résultat est enregistré. Elle se lit comme une table, mais Redshift la met à jour lui-même lors du rafraîchissement. Les indicateurs de revenus quotidiens, hebdomadaires et mensuels demandés par le sujet sont ainsi disponibles directement :

```
CREATE MATERIALIZED VIEW mv_revenue_monthly AS
SELECT d.year,
       d.month,
       a.merchant_key,
       SUM(a.total_amount_usd)  AS revenus_usd,
       SUM(a.refund_amount_usd) AS remboursements_usd,
       SUM(a.transaction_count) AS nb_transactions
FROM agg_revenue_daily a
JOIN dim_date d ON a.date_key = d.date_key
GROUP BY d.year, d.month, a.merchant_key;

REFRESH MATERIALIZED VIEW mv_revenue_monthly;
```

4. Jointures à grande échelle

Redshift répartit les données sur plusieurs nœuds. Une jointure est rapide quand les lignes à relier sont sur le même nœud. La répartition est donc choisie table par table :

- **Clé de distribution :** fact_transaction et dim_customer sont réparties selon customer_key. Les lignes d'un même client sont ainsi sur le même nœud et la jointure se fait sans échange entre nœuds.
- **Petites dimensions copiées partout :** dim_date, dim_country, dim_currency et dim_payment_method sont copiées en entier sur chaque nœud (distribution ALL). Elles sont petites, et toutes les jointures avec elles restent locales.
- **Clé de tri :** les tables de faits et les tables pré-agrégées sont triées par date_key. Comme presque toutes les requêtes filtrent sur une période, Redshift ne lit que les blocs de données de cette période et ignore les autres.
- **Stockage en colonnes :** Redshift ne lit que les colonnes utilisées par la requête et les compresse, ce qui réduit fortement le volume lu.

5. Sous-requêtes

- **Filtrer tôt :** les filtres sur la date sont placés au plus près des tables de faits, afin de réduire le nombre de lignes avant les jointures.
- **Requêtes en étapes (CTE, Common Table Expression) :** une requête complexe est découpée avec WITH en étapes lisibles, plutôt qu'en sous-requêtes imbriquées qui seraient recalculées pour chaque ligne.
- **Résultats intermédiaires enregistrés :** un calcul lourd utilisé souvent, comme les indicateurs RFM de chaque client (livrable 8, requête 5), est fait une fois par semaine et son résultat est enregistré (segment dans dim_customer), au lieu d'être recalculé dans chaque requête.

6. Analyse des séries temporelles

La table dim_date décrit chaque jour (jour, semaine, mois, trimestre, année, jour de la semaine, week-end) et fact_transaction contient l'heure de la transaction (hour_of_day). Cela permet de regrouper les données à n'importe quel niveau de temps. Les fonctions de fenêtre de SQL permettent ensuite de comparer une période à la précédente, par exemple pour calculer l'évolution des revenus d'un mois sur l'autre :

```
SELECT year,
       month,
       SUM(revenus_usd) AS revenus_usd,
       ROUND(100.0 * (SUM(revenus_usd) - LAG(SUM(revenus_usd)) OVER (ORDER BY year, month))
             / LAG(SUM(revenus_usd)) OVER (ORDER BY year, month), 1) AS evolution_pct
FROM mv_revenue_monthly
GROUP BY year, month
ORDER BY year, month;
```

Ces séries servent aussi au modèle de prévision des revenus (livrable 7).

7. Données historiques et quasi temps réel

- **Historique :** les lignes de fact_transaction ne sont jamais supprimées. Les analyses peuvent donc porter sur plusieurs années, même pour des transactions retirées de l'OLTP après 24 mois.
- **Quasi temps réel :** l'ELT charge les nouvelles données toutes les heures. Les tableaux de bord affichent donc des données de moins d'une heure, comme fixé dans les exigences du livrable 0. Les besoins vraiment temps réel (fraude) sont traités par Redis et l'OLTP, pas par le data warehouse.
- **Montée en charge :** des nœuds peuvent être ajoutés à Redshift quand le volume augmente, et de la capacité temporaire est ajoutée automatiquement lors des pics de requêtes (concurrency scaling).

8. Conformité

Le data warehouse ne contient aucune donnée personnelle directe : les clients y sont identifiés uniquement par leur identifiant (livrable 6). La table fact_audit reprend le journal d'audit de l'OLTP et permet de produire automatiquement des rapports sur les accès aux données clients (livrable 8, requête 16).